Finally, check the result for the readiness probe endpoint:

```
$ http :9001/actuator/health/readiness
{
  "status": "UP"
}
```

The readiness state of a Spring Boot application indicates whether it's ready to accept traffic and process new requests. During the startup phase or graceful shutdown, the application is not ready and will refuse any requests. It might also become temporarily not ready if, at some point, it's overloaded. When it's not ready, Kubernetes will not send any traffic to the application instance.

When you're done testing the health endpoints, stop the application (Ctrl-C) and the containers (`docker-compose down`).

**NOTE** Go ahead and add Spring Boot Actuator to all the applications composing the Polar Bookshop system. In Order Service and Edge Service, remember to configure unauthenticated access to the Actuator endpoints in the `SecurityConfig` class, as we did for Catalog Service. In Dispatcher Service you'll also need to add a dependency on Spring WebFlux (`org.springframework.boot:spring-boot-starter-webflux`) because Actuator needs a web server configured to serve its endpoints over HTTP. Then configure the health endpoints for all the applications, as you learned in this section. As a reference, you can look at the source code repository accompanying this book (Chapter13/13-end).

By default, the readiness probe in Spring Boot doesn't depend on any external components. You can decide whether any external systems should be included in the readiness probe.

For example, Catalog Service is an external system for Order Service. Should you include it in the readiness probe? Since Order Service adopts resilience patterns to deal with the scenario where Catalog Service is unavailable, you should keep Catalog Service out of the readiness probe.